Contents

Introduction to Modern Data Automation 3
The evolution from Legacy Excel to the Power Suite. 3
Understanding the "M" Engine vs. standard Excel formulas. 3
Connecting to Data Sources 4
Importing data from Folders (Batch processing). 4
Connecting to Web data and SQL databases. 4
Establishing live links vs. static imports. 5
The Power Query Editor 5
The Ribbon and the "Applied Steps" pane (The Undo/Redo of Automation). 5
Data types: Why they matter for automation. 5
Filtering, sorting, and removing duplicates at the source. 6
Basic Transformations 6
Splitting columns by delimiter and positions. 6
Merging and appending queries (The VLOOKUP killer). 7
Using "From Selection" to create custom columns. 7
Advanced Data Shaping 8
Pivoting and Unpivoting data for reporting. 8
Conditional columns and custom logic. 8
Grouping data for aggregate summaries. 9
Copilot in Excel: The AI Assistant 9
Using Copilot to write complex formulas from natural language. 9
AI-driven data exploration and highlighting trends. 10
Generating "What-if" scenarios with AI prompts. 10
Cleaning "Dirty" Data with AI 10
Using Copilot to identify inconsistencies in datasets. 10
Automating text cleanup (Proper casing, trimming, and prefix removal). 11
Parameters and Functions 11
Creating dynamic file paths with parameters. 11
Turning a query into a reusable custom function. 12

Loading Data & The Data Model 13
Loading to Tables vs. PivotCharts. 13
Introduction to the Data Model (Power Pivot). 13
Understanding Relationships (One-to-Many). 13
Dynamic Reporting with Copilot 14
Asking Copilot to create PivotTable layouts. 14
Generating automatic Chart suggestions via AI. 14
Building "One-Click" Refresh dashboards. 15
Error Handling & Optimization 15
Identifying and fixing Power Query "Privacy Levels." 15
Managing Refresh errors and broken file paths. 16
Capstone Project: End-to-End Automation 17
Project: Build a self-updating Monthly Sales Report. 17
Automating the consolidation of 12 monthly CSV files. 17
Presenting findings using Copilot-generated insights. 17

Introduction to Modern Data Automation

The evolution from Legacy Excel to the Power Suite.

Legacy Excel relies on manual copy-pasting, localized grid formulas (like VLOOKUP), and physical file storage. The Power Suite (Power Query and Power Pivot) shifts this to automated, repeatable data pipelines capable of handling millions of rows without bloating the file size.

Activity 1: Bypassing the Row Limit.

1. Open a blank Excel workbook.
2. Instead of trying to paste 300,000 rows directly into the grid (which would slow Excel to a crawl), go to **Data > Get Data > From File > From Folder**.
3. Point to your C:\PowerQueryDemo folder to load the metadata of all three files instantly.

Activity 2: File Size Optimization.

1. After connecting to the Sales_Transactions.csv, select **Load To...** instead of Load.
2. Choose **Only Create Connection** and check **Add this data to the Data Model**.
3. Save the Excel file. Notice it remains under 1MB despite "holding" 100,000 rows.

Activity 3: Automating Repetitive Updates.

1. Once the folder connection is established, simulate next month's data by adding a new CSV to the folder.
2. In Excel, navigate to **Data > Refresh All**.
3. Power Query will automatically ingest the new file without any manual copy-pasting.

Understanding the "M" Engine vs. standard Excel formulas.

Standard Excel formulas calculate on the visual grid. The "M" Engine is the functional language operating *behind* Power Query, processing data step-by-step before it ever reaches the Excel grid.

Note: To test the "M" code examples below, open the Power Query Editor in Excel (**Data > Get Data > Launch Power Query Editor**). Select your query, go to the **Add Column** tab on the Ribbon, and click **Custom Column**. Paste the formula into the dialog box.

Activity 4: Extracting Data Elements.

1. **Excel Approach:** In a grid, you would use =YEAR(B2) to get the year from OrderDate.
2. **M Engine Approach:** In Power Query, add a Custom Column and type: Date.Year([OrderDate]).

Activity 5: Conditional Logic.

1. **Scenario:** Flagging high-value sales.
2. **Excel Approach:** =IF(I2>100, "High", "Standard")
3. **M Engine Approach:** In a Custom Column, type: if [UnitPrice] > 100 then "High" else "Standard".

Activity 6: Handling Text Cases.

1. **Scenario:** Ensuring Senator names (like "Risa" or "Raffy") are uniformly capitalized.

2. **M Engine Approach:** Instead of writing a complex nested formula, select the CustomerName column, go to **Transform > Format > Uppercase**. The M code Text.Upper([CustomerName]) is automatically generated in the background.

Connecting to Data Sources

Importing data from Folders (Batch processing).

This feature allows you to point Excel to a specific directory. It will ingest, clean, and stack (append) all compatible files it finds there automatically.

Activity 7: Establishing the Folder Connection.

1. Navigate to **Data > Get Data > From File > From Folder**.
2. Browse to the folder where you saved the generated CSVs (C:\PowerQueryDemo) and click **Open**.
3. Click **Transform Data** to open the Power Query Editor.

Activity 8: Filtering for Specific Files.

1. In the Power Query Editor, locate the Extension column.
2. Click the drop-down filter and ensure only .csv is selected.
3. Locate the Name column and filter to "Text Contains" the word "Sales" to isolate just the transaction files.

Activity 9: Combining the Binaries.

1. Locate the Content column (which contains the binary data of the files).
2. Click the **Combine Files** icon (two downward arrows) in the column header.
3. Power Query will automatically extract and stack the 100,000 rows.

Connecting to Web data and SQL databases.

Power Query can fetch live data from external servers, APIs, or websites to enrich your local files.

Activity 10: Web Import for Currency Conversion.

1. Go to **Data > Get Data > From Other Sources > From Web**.
2. Enter the URL of a public exchange rate table (e.g., PHP to USD).
3. Power Query will parse the webpage; select the relevant HTML table and click **Transform Data** to merge it with your TotalAmount column.

Activity 11: Connecting to SQL.

1. Go to **Data > Get Data > From Database > From SQL Server Database**.
2. Enter your server name and database name.
3. Select the target table and import it directly into the Power Query Editor to join with your CSV data.

Activity 12: Merging Web Data with Local Data.

1. With your web query (Exchange Rates) and local query (Sales_Transactions) loaded, select the Sales query.
2. Go to **Home > Merge Queries**.
3. Match the common keys (e.g., Currency Type) to pull the live exchange rate into your sales dataset.

Establishing live links vs. static imports.

A static import is a one-time copy of data. A live link establishes a query path that can be refreshed with a single click to retrieve the latest data.

Activity 13: Configuring Background Refresh.

1. In Excel, click **Queries & Connections** on the Data ribbon.
2. Right-click your Sales_Transactions query and select **Properties**.
3. Check the box for **Enable background refresh** so users can continue working while data updates.

Activity 14: Setting Time-Based Automation.

1. In the same **Properties** window, check **Refresh every [X] minutes**.
2. Set the timer to 60 minutes for near-live updates from your source folder.

Activity 15: Refresh on Open.

1. Still in **Properties**, check **Refresh data when opening the file**.
2. This guarantees the dashboard always shows the latest data the moment the workbook is accessed.

The Power Query Editor

The Ribbon and the "Applied Steps" pane (The Undo/Redo of Automation).

Activity 16: Tracking Transformations.

1. In Power Query, select the CustomerName column in the Customer_Interactions.csv.
2. Go to **Transform > Format > Trim**.
3. Look at the Applied Steps pane on the right; a new step called "Trimmed Text" has appeared.

Activity 17: Time Travel (Undo).

1. Select the TotalAmount column and press the Delete key.
2. To undo this, do *not* press Ctrl+Z. Instead, go to the Applied Steps pane.
3. Click the red "X" next to the "Removed Columns" step. The data instantly reappears.

Activity 18: Inspecting Past States.

1. Click on an older step in the Applied Steps list (e.g., "Source").
2. The main window will temporarily revert to show how the data looked at that exact moment in the pipeline.
3. Click back to the final step to return to the present state.

Data types: Why they matter for automation.

Assigning precise data types ensures functions operate correctly. You cannot perform math on a text string, nor extract a month from a raw string.

Activity 19: Date/Time Conversion.

1. Look at the InteractionDate column; it likely shows a date and a time.
2. Click the icon in the column header (it usually says "ABC/123").
3. Select **Date**. The time portion is stripped, allowing for clean daily aggregations.

Activity 20: Decimal/Currency Enforcement.

1. Select the BudgetAllocated column in the Marketing_Campaigns dataset.
2. Click the data type icon and select **Currency**.
3. This locks the format for any subsequent Power Pivot calculations.

Activity 21: Boolean Transformations.

1. In the Sales dataset, locate the IsReturned column ("Yes"/"No").
2. Go to **Transform > Replace Values**. Replace "Yes" with "true" and "No" with "false".
3. Change the data type to **True/False**. This processes much faster in the engine than text strings.

Filtering, sorting, and removing duplicates at the source.

It is a best practice to remove unnecessary rows and columns as early as possible in your Applied Steps to reduce memory consumption and speed up processing.

Activity 22: Early Filtering.

1. In the Sales_Transactions query, click the drop-down on the Region column.
2. Uncheck all, and select only "NCR".
3. This immediately drops the dataset size before subsequent heavy processing steps occur.

Activity 23: Removing Duplicates.

1. Select the TransactionID column.
2. Right-click the column header and choose **Remove Duplicates**.
3. This guarantees a clean dataset, especially useful if appending multiple files resulted in overlap.

Activity 24: Sorting for Top N Analysis.

1. Select the TotalAmount column.
2. Click the drop-down and select **Sort Descending**.
3. You can then go to **Home > Keep Rows > Keep Top Rows** and type "100" to instantly create a "Top 100 Transactions" dataset.

Basic Transformations

Splitting columns by delimiter and positions.

Real-world data often crams multiple pieces of information into one cell. Power Query allows you to split this data systematically without messy LEFT(), RIGHT(), or FIND() formulas.

Activity 25: Splitting by Delimiter.

1. In the Sales_Transactions query, locate the WarehouseID column (e.g., "WH-01").
2. Right-click the column header, select **Split Column > By Delimiter**.
3. Choose the hyphen (-) and click **OK**. You now have a "Warehouse Prefix" and "Warehouse Number" column.

Activity 26: Splitting by Position.

1. Select the TransactionID column (make sure its data type is set to Text first).
2. Right-click and choose **Split Column > By Number of Characters**.
3. Enter 3 and choose **Once, as far left as possible**. This isolates the first three digits of every transaction.

Activity 27: Splitting by Digit to Non-Digit Transition.

1. Select the PromoCode column (e.g., "SUMMER20").
2. Right-click and choose **Split Column > By Non-Digit to Digit**.
3. This cleanly separates the text "SUMMER" from the number "20" into two distinct columns.

Merging and appending queries (The VLOOKUP killer).

VLOOKUP and XLOOKUP are notoriously slow on 100,000-row datasets. Power Query uses join operations (Merge) to combine columns across tables, and Append to stack tables on top of each other vertically.

Activity 28: The Inner Join (Merge).

1. With Sales_Transactions open, go to **Home > Merge Queries**.
2. Select Customer_Interactions as the second table.
3. Click on the CustomerName column in both tables (e.g., linking records for Senator Risa or Senator Raffy) and select **Inner Join**. This filters the list to only customers who both bought an item *and* contacted support.

Activity 29: The Left Anti Join (Finding Exceptions).

1. Repeat the merge steps above, but change the Join Kind to **Left Anti**.
2. This instantly identifies customers who made a purchase but have *never* interacted with customer service.

Activity 30: Appending Data.

1. If you had split your marketing data into "2025 Campaigns" and "2026 Campaigns", you would go to **Home > Append Queries**.
2. Select both tables. Power Query stacks them vertically, perfectly aligning headers like CampaignID and Platform.

Using "From Selection" to create custom columns.

Also known as "Column From Examples," this uses AI within Power Query to detect patterns based on examples you type, writing the complex "M" code for you.

Activity 31: Text Extraction.

1. In Marketing_Campaigns, go to **Add Column > Column From Examples > From Selection**.
2. Select the Platform column. If a cell says "Google Ads", type "Google" in the new example column.
3. Press Enter. Power Query detects the pattern and extracts the first word for all 100,000 rows.

Activity 32: Combining Text.

1. Select the CustomerName column in the sales dataset.
2. Open **Column From Examples**. If the name is "Grace", type "Senator Grace".
3. Power Query automatically generates the logic to prepend "Senator " to every name in the dataset.

Activity 33: Date Reformatting.

1. Select OrderDate. Open **Column From Examples**.
2. If the date is "2025-01-05", type "Jan-2025".
3. Power Query will write the necessary date extraction and formatting M code automatically.

Advanced Data Shaping

Pivoting and Unpivoting data for reporting.

Unpivoting transforms "wide" data (many columns) into "long" data (fewer columns, more rows), which is the optimal format for building PivotTables and Data Models.

Activity 34: Unpivoting Specific Columns.

1. In Marketing_Campaigns, hold CTRL and select the Impressions, Clicks, and Conversions columns.
2. Right-click any of the selected headers and choose **Unpivot Columns**.
3. You now have an "Attribute" column (containing the metric names) and a "Value" column (containing the numbers), making it easier to chart all three metrics in a single visual.

Activity 35: Unpivot Other Columns (Future-proofing).

1. Select the CampaignID column.
2. Right-click and choose **Unpivot Other Columns**.
3. If new metrics are added to the CSV next month, this query will automatically unpivot them without breaking.

Activity 36: Pivoting Data.

1. Select your new "Attribute" column (the one with the metric names).
2. Go to **Transform > Pivot Column**.
3. Choose your "Value" column for the values and select "Sum" under advanced options. The data returns to its original wide state.

Conditional columns and custom logic.

This feature allows you to build robust IF/THEN/ELSE logic using a simple graphical interface.

Activity 37: Creating Spend Tiers.

1. In Marketing_Campaigns, go to **Add Column > Conditional Column**.
2. Name it "Spend Tier". Set logic: IF AmountSpent is greater than 40000 THEN "High", ELSE IF AmountSpent is greater than 10000 THEN "Medium", ELSE "Low".

Activity 38: Flagging Urgent Support Tickets.

1. In Customer_Interactions, open the **Conditional Column** dialog.
2. Set logic: IF ResolutionStatus equals "Pending" THEN check a secondary condition (Wait, the UI doesn't support nested AND directly in one line).
3. *Alternative:* If EscalationTier equals "Tier 3" THEN "Urgent", ELSE "Normal".

Activity 39: Testing Custom "M" Code Logic.

- *Where to test:* In the Power Query Editor, go to **Add Column > Custom Column**.
- *How to test:* Type = if [LoyaltyTier] = "Platinum" and [NPS_Score] < 5 then "At Risk" else "Safe". This demonstrates how to combine conditions using raw M code.

Grouping data for aggregate summaries.

Grouping reduces millions of rows into summarized, high-level tables, which drastically reduces file size if you only need the aggregate data for your dashboard.

Activity 40: Simple Aggregation.

1. In Sales_Transactions, go to **Transform > Group By**.
2. Group by Region. Set the New Column Name to "Total Sales", Operation to "Sum", and Column to TotalAmount.

Activity 41: Multi-Level Grouping.

1. Open **Group By** and select **Advanced**.
2. Group by CustomerName (e.g., Sherwin, Imee) and then by Category.
3. Add aggregations to calculate both "Total Quantity" (Sum of Quantity) and "Average Price" (Average of UnitPrice).

Activity 42: Grouping with All Rows.

1. Open **Group By**, group by SalesRep.
2. For the operation, choose **All Rows**.
3. This nests the underlying tables inside a single column, allowing you to perform advanced row-level calculations within those specific groups later.

Copilot in Excel: The AI Assistant

Using Copilot to write complex formulas from natural language.

Copilot translates plain English requests into complex DAX or Excel formulas, lowering the barrier to entry for advanced analytics.

Note: Close and Load your data from Power Query to the Excel Data Model or a Table. Click the **Copilot** button on the Excel Home Ribbon.

Activity 43: Formula Generation.

1. Open the Copilot pane and type: *"Add a column that calculates the profit margin by subtracting 20% from the TotalAmount as cost of goods sold."*
2. Copilot will generate the exact Excel formula and offer an "Insert Column" button.

Activity 44: Date Calculations.

1. Type: *"Create a formula to calculate the number of days between OrderDate and DeliveryDate."*
2. Copilot will generate =DAYS([@DeliveryDate], [@OrderDate]) and explain how it works.

Activity 45: Text Manipulation.

1. Type: *"Write a formula to extract only the first name from the CustomerName column."*
2. Copilot will output the correct combination of LEFT and SEARCH or FIND functions.

AI-driven data exploration and highlighting trends.

Instead of manually building charts to find insights, Copilot can scan the dataset and automatically surface anomalies, trends, and patterns.

Activity 46: Spotting Trends.

1. In the Copilot pane, prompt: *"Are there any seasonal trends in the OrderDate data across different Regions?"*
2. Copilot will analyze the dates and output a summary (e.g., "Sales in NCR peak during Q4").

Activity 47: Outlier Detection.

1. Prompt: *"Identify any campaigns in the Marketing_Campaigns table that have an unusually high CostPerClick compared to the average."*
2. Copilot will list the specific CampaignIDs and their costs.

Activity 48: Auto-Charting.

1. Prompt: *"Show me a visualization of RevenueGenerated by Platform."*
2. Copilot will generate a PivotChart directly in the pane. Click "Add to a new sheet" to instantly build your dashboard.

Generating "What-if" scenarios with AI prompts.

Copilot can act as an analytical partner, helping you project future outcomes based on historical data patterns without building complex financial models.

Activity 49: Budget Reallocation.

1. Prompt Copilot: *"What if we shifted 20% of the BudgetAllocated from 'Awareness' campaigns to 'Conversion' campaigns? Estimate the new Conversions based on the current conversion rates."*
2. Copilot will calculate the hypothetical math and present the projected outcome.

Activity 50: Pricing Strategy.

1. Prompt: *"If we increased the UnitPrice by 5% across all 'Electronics' category items, how much additional TotalAmount would we generate, assuming Quantity stays the same?"*

Activity 51: Operational Impact.

1. Prompt: *"Analyze the Customer_Interactions dataset. If we reduce FirstResponseTimeMin by 50%, predict the impact on the SatisfactionScore based on existing correlations in the data."*

Cleaning "Dirty" Data with AI

Using Copilot to identify inconsistencies in datasets.

AI can quickly spot data quality issues without manual filtering. Copilot acts as an automated auditor, saving you the time of visually scanning thousands of rows for anomalies.

Activity 52: Spotting Categorical Errors.

1. Load the Sales_Transactions dataset into an Excel table.
2. Open the Copilot pane and prompt: *"Review the Category column. Are there any categories that appear to be typos or fall outside the expected main groups?"*

3. Copilot will analyze the unique values and flag inconsistencies.

Activity 53: Identifying Date Anomalies.

1. Prompt Copilot: *"Check the OrderDate and DeliveryDate columns. Are there any records where the DeliveryDate occurs before the OrderDate?"*
2. Copilot will instantly generate a list or a filtered view of these logical impossibilities.

Activity 54: Finding Extreme Outliers.

1. Prompt Copilot: *"Analyze the SessionDurationMin in the Sales data. Highlight any extreme outliers that might indicate a tracking error."*
2. Copilot will pinpoint durations that statistically deviate from the norm.

Automating text cleanup (Proper casing, trimming, and prefix removal).

Power Query excels at standardizing text formatting automatically upon import. This ensures consistency before the data ever reaches your reporting layer.

Activity 55: Standardizing Names.

1. In Power Query, select the CustomerName column (e.g., if a Senator's name was typed as "risa" or "RISA").
2. Go to **Transform > Format > Capitalize Each Word**.
3. This standardizes all text entries regardless of how the raw CSV was typed.

Activity 56: Trimming Whitespace.

1. Select the ReturnReason column in the Sales dataset.
2. Go to **Transform > Format > Trim**.
3. This automatically removes invisible leading or trailing spaces that would otherwise break PivotTable groupings.

Activity 57: Prefix Removal.

1. Select the WarehouseID column (e.g., "WH-01").
2. Go to **Transform > Replace Values**.
3. Enter "WH-" in the "Value To Find" box, leave "Replace With" blank, and click OK to strip the prefix and leave only the numbers.

Parameters and Functions

Creating dynamic file paths with parameters.

Hardcoded file paths break when sharing files. Parameters make the source directory dynamic, allowing you to hand the workbook to a colleague without the queries failing.

Activity 58: Creating the Parameter.

1. In Power Query, go to **Home > Manage Parameters > New Parameter**.
2. Name it FolderPath, set Type to "Text", and set the Current Value to your directory (e.g., C:\PowerQueryDemo\).

Activity 59: Applying the Parameter.

1. Go to the "Source" step of your Sales_Transactions query in the Applied Steps pane.

2. Click the gear icon next to "Source".
3. Change the file path dropdown from "Text" to "Parameter", and select your FolderPath parameter.

Activity 60: Testing Dynamic Updates.

1. Close & Load the query.
2. If you move the CSVs to a new folder, you only need to update the FolderPath parameter value once, and all three datasets will successfully reconnect.

Turning a query into a reusable custom function.

If you have complex transformation logic, you can save it as a function (fx) to apply across multiple queries.

Note: **Where to place and test this code:** In the Power Query Editor, go to **Home > Advanced Editor** for a blank query to paste the code below, or use the UI steps to generate it. To test it, go to **Add Column > Invoke Custom Function**.

Activity 61: Defining the Logic.

1. Goal: Create a function that calculates a dynamic net score based on sentiment and satisfaction.
2. Create a blank query and open the Advanced Editor.

Activity 62: Writing the M Code Function.

1. Type the following code:

```
(Satisfaction as number, Sentiment as number) =>
let
  NetScore = (Satisfaction * 10) + (Sentiment * 20)
in
  NetScore
```

2. Name this query fxCalculateNetScore.

Activity 63: Invoking the Function.

1. Go to the Customer_Interactions query.
2. Go to **Add Column > Invoke Custom Function**.
3. Select fxCalculateNetScore, and map the SatisfactionScore and SentimentScore columns to the function's arguments. It will calculate the result row-by-row.

Loading Data & The Data Model

Loading to Tables vs. PivotCharts.

Choosing the right destination prevents file bloat and optimizes dashboard performance.

Activity 64: Loading to a Table.

1. When finishing your Power Query work on the Marketing dataset, click **Close & Load To...**
2. Select **Table**. This physically writes the 100,000 rows onto the Excel grid. (Good for row-level auditing, bad for file size).

Activity 65: Loading to a PivotChart directly.

1. Click **Close & Load To...** and select **PivotChart**.
2. This bypasses the grid entirely, loading the visual directly from the engine, keeping the workbook lightweight.

Activity 66: Connection Only.

1. Click **Close & Load To...** and select **Only Create Connection**.
2. Check **Add this data to the Data Model**. This is the optimal route for massive datasets.

Introduction to the Data Model (Power Pivot).

The Data Model compresses large datasets and allows you to link multiple tables together, breaking the 1-million row Excel limit.

Activity 67: Enabling the Data Model.

1. Ensure all three datasets (Sales_Transactions, Marketing_Campaigns, Customer_Interactions) are loaded as "Connection Only" and added to the Data Model.

Activity 68: Accessing Power Pivot.

1. Go to the **Power Pivot** tab on the Excel Ribbon (you may need to enable this add-in via Options).
2. Click **Manage** to open the Data Model engine window.

Activity 69: Creating a DAX Measure.

1. In the Power Pivot window, select the Sales_Transactions tab.
2. Click in the calculation area at the bottom and type: Total Sales Revenue := SUM([TotalAmount]). This creates a portable calculation you can use in any PivotTable.

Understanding Relationships (One-to-Many).

Relationships replace VLOOKUPs by linking primary keys (the "One" side) to foreign keys (the "Many" side).

Activity 70: Opening Diagram View.

1. Inside the Power Pivot window, click **Diagram View** on the Home ribbon. You will see your tables represented as boxes.

Activity 71: Building the Link.

1. Click and drag the CustomerName field from the Customer_Interactions table to the CustomerName field in the Sales_Transactions table.

2. A line will appear, linking Senator names (e.g., Senator Raffy's purchases linked to his support tickets).

Activity 72: Testing the Relationship.

1. Create a new PivotTable from the Data Model.
2. Put IssueType (from Interactions) in the Rows, and TotalAmount (from Sales) in the Values. You can now see the total sales revenue generated by customers who complained about specific issues.

Dynamic Reporting with Copilot

Asking Copilot to create PivotTable layouts.

Instead of dragging and dropping fields, you can instruct Copilot to build the PivotTable structure for you.

Activity 73: Automated Cross-Tabulation.

1. Open the Copilot pane in Excel.
2. Prompt: *"Create a PivotTable showing TotalAmount by Region and Category from the Sales dataset."*
3. Copilot will instantly build and insert the PivotTable.

Activity 74: Time Intelligence via AI.

1. Prompt: *"Build a PivotTable that breaks down RevenueGenerated by Month using the StartDate in the Marketing dataset."*

Activity 75: AI Filtering.

1. Prompt: *"Update the PivotTable to only show data where the Interaction ResolutionStatus is 'Escalated'."*

Generating automatic Chart suggestions via AI.

Copilot analyzes the PivotTable or Data Model and recommends the best visual representation.

Activity 76: Generating a Trendline.

1. Select your Sales by Month PivotTable.
2. Prompt Copilot: *"Insert a chart that best visualizes this sales trend over time."* Copilot will generate a line chart.

Activity 77: Visualizing Comparisons.

1. Prompt Copilot: *"Show me a visual comparing the BudgetAllocated vs. AmountSpent by Platform."*
2. Copilot will generate a clustered column chart comparing the two metrics side-by-side.

Activity 78: Iterative Design.

1. If Copilot generates a pie chart for PaymentMethod, prompt it: *"Please change this pie chart to a horizontal bar chart."*

Building "One-Click" Refresh dashboards.

The ultimate goal of automation is connecting the raw CSVs, the Power Query transformations, the Data Model, and the visual dashboard so they all update with a single click.

Activity 79: Assembling the Dashboard.

1. Cut and paste your Copilot-generated charts (Sales Trend, Marketing Spend, Support Issues) onto a clean, blank Excel worksheet tab named "Dashboard".

Activity 80: Connecting Slicers.

1. Select one chart, go to **PivotChart Analyze > Insert Slicer**, and choose Region.
2. Right-click the Slicer, go to **Report Connections**, and check all PivotTables. Now, clicking "NCR" filters the entire dashboard.

Activity 81: The "One-Click" Test.

1. Simulate new data by opening your raw Sales_Transactions.csv in notepad and duplicating a few rows at the bottom, then save.
2. Go to your Excel Dashboard and click **Data > Refresh All**.
3. Watch as Power Query imports the new rows, updates the Data Model, and refreshes all dashboard charts instantly.

Error Handling & Optimization

Identifying and fixing Power Query "Privacy Levels."

Power Query includes a data privacy firewall to prevent you from accidentally sending sensitive local data (like Senator Risa's purchase history in your Sales_Transactions.csv) to a public web server when merging queries. If privacy levels clash or aren't defined, the engine will block the refresh to protect you.

Activity 82: Encountering the Formula.Firewall Error.

1. Attempt to merge your local Sales_Transactions query with a live Web API query (like a public exchange rate table).
2. When you click **Close & Load**, a yellow banner or popup will demand you specify data privacy levels. If ignored or set incorrectly, a Formula.Firewall error halts the process.

Activity 83: Configuring Source Permissions.

1. Go to **Data > Get Data > Data Source Settings**.
2. Select the web API from the list and click **Edit Permissions**. Set the Privacy Level to **Public**.
3. Select your local folder (C:\PowerQueryDemo) and set it to **Private** or **Organizational**. This tells the engine it is safe to pull data *from* the public web, but not safe to send local data *to* the web.

Activity 84: Bypassing the Firewall (For Local Training Scenarios).

1. In Excel, go to **Data > Get Data > Query Options**.
2. Under **Current Workbook > Privacy**, select **Ignore the Privacy Levels and potentially improve performance**.
3. *Note for students:* Only do this when you are absolutely certain no sensitive data is being leaked via a web call.

Managing Refresh errors and broken file paths.

Hardcoded transformation steps are fragile. If the underlying Customer_Interactions.csv changes a column name, or if the file is moved to a new drive, the entire automation breaks. Knowing how to quickly diagnose and fix these errors is crucial.

Where to place and test this code: Open the Power Query Editor, go to the **View** tab, and click **Advanced Editor**.

How to test it: Locate the step where you are removing or renaming a column, and replace the standard "M" code with the robust snippet provided in Demo 2.

Activity 85: Fixing a Broken File Path (Data Source Error).

1. *Simulate the error:* Close Excel, rename your C:\PowerQueryDemo folder to C:\PowerQueryArchive, and reopen Excel. Click **Refresh All**. Excel will throw a "DataFormat.Error: Could not find a part of the path".
2. *The Fix:* Go to **Data > Get Data > Data Source Settings**. Select the broken file path, click **Change Source**, and browse to the new folder name. (Alternatively, remind them of the Parameter solution from Topic 20).

Activity 86: Handling Missing Columns Gracefully.

1. *Scenario:* You built a step that removes the LoyaltyTier column. Next month, the IT team stops including that column in the CSV. The query fails with: "The column 'LoyaltyTier' of the table wasn't found."
2. *The Fix (M Code):* In the Advanced Editor, find the step: Table.RemoveColumns(Source, {"LoyaltyTier"}).
3. Modify it to: Table.RemoveColumns(Source, {"LoyaltyTier"}, MissingField.Ignore). Adding MissingField.Ignore tells the engine to just skip the instruction if the column is already gone, saving the refresh from crashing.

Activity 87: The "Go To Error" Feature for Row-Level Issues.

1. *Simulate the error:* Open the raw Sales_Transactions.csv in Notepad, find Senator Raffy's record, and replace his numeric UnitPrice with the text "N/A". Save the file.
2. Refresh Excel. It will load the data but display a warning: "Errors in Sales_Transactions".
3. Clicking the "Errors" hyperlink generates a new, temporary query folder that isolates only the rows containing errors. This allows you to instantly spot the "N/A" text in the numeric column and either fix the source file or add a "Replace Errors" step in your query.

Capstone Project: End-to-End Automation

Project: Build a self-updating Monthly Sales Report.

Automating the consolidation of 12 monthly CSV files.

This project simulates a real-world request from executive stakeholders (like Senator Grace or Senator Francis). The goal is to build a self-updating Monthly Sales Report. Instead of manually opening and pasting 12 different files together every time the year ends, students will build a pipeline that does the heavy lifting automatically. Because your laptop uses a SATA hard disk, this will also perfectly demonstrate how Power Query efficiently handles bulk file processing without freezing the machine.

Activity 88: Automating the consolidation of 12 monthly CSV files.

1. Open a blank Excel workbook.
2. Go to **Data > Get Data > From File > From Folder** and point it to the C:\MonthlySales directory.
3. Click **Transform Data**.
4. In the Power Query Editor, click the **Combine Files** icon (the two downward arrows) on the Content column.
5. Power Query will automatically stack all 12 months of 2025 data into a single, cohesive table.

Activity 89: Applying the Transformation Rules.

1. Rename the query to Consolidated_2025_Sales.
2. Apply the cleanup steps learned earlier: ensure OrderDate is set to the Date type, capitalize the CustomerName column, and add a Custom Column calculating Profit (e.g., [TotalAmount] * 0.15).
3. Click **Close & Load To...** and choose **PivotChart** (or **Only Create Connection** and add to the Data Model to save memory).

Activity 90: The "Self-Updating" Magic.

1. To prove the automation works, have the students copy a file from the folder, rename it to Sales_2026_01.csv, and paste it back into the C:\MonthlySales folder.
2. In Excel, simply click **Data > Refresh All**. The new file is instantly ingested, processed, and added to the underlying data model without a single VLOOKUP.

Presenting findings using Copilot-generated insights.

A dashboard is only as good as the insights it provides. In this final phase, students will use Copilot to generate the visual layout and extract narrative insights to present to their mock stakeholders.

Activity 91: AI-Generated Visualizations.

1. Open the **Copilot** pane on the Excel Home ribbon.
2. Prompt: *"Based on the Data Model, create a PivotChart showing the TotalAmount by Region, and another showing the top 5 Category items by Quantity sold."*
3. Copilot will generate the charts. Click "Add to a new sheet" to begin assembling the final dashboard view.

Activity 92: Extracting Narrative Insights.

1. Executives often want a written summary alongside the charts.

2. Prompt Copilot: *"Analyze the consolidated sales data and provide a 3-bullet-point executive summary highlighting the most profitable Region and the most common ReturnReason."*
3. Copy Copilot's response and paste it into a text box at the top of the dashboard.

Activity 93: Finalizing the Dashboard.

1. Insert Slicers for Region and Category.
2. Connect the Slicers to all AI-generated PivotCharts using **Report Connections**.
3. The students now have a fully interactive, self-updating dashboard complete with AI-driven narrative analysis.